

Execution-Verified AI Feedback for Code Generation: An Implementation and Incomplete Evaluation

Anonymous Authors¹

Abstract

Code generation models can be improved through post-training feedback, but existing methods face a trade-off: execution feedback (test pass/fail) provides ground-truth correctness but no semantic guidance, while AI feedback (LLM critique) offers explanations but is incorrect approximately 61% of the time according to prior analysis. This work proposes Execution-Verified AI Feedback (EVAF), a mechanism that filters AI-generated code critiques through unit test execution before using them as training signals. By accepting only AI suggestions that pass tests, EVAF aims to combine execution’s fidelity with AI’s semantic richness. An implementation was developed using CodeT5-770M as the baseline generator and CodeLlama-7b-Instruct as the feedback model, targeting HumanEval (164 problems). However, experimental validation did not complete: the pipeline terminated at approximately 53% progress (78-87 of 164 problems processed in the EVAF gating phase) before any aggregate metrics could be computed. The experiment logs show baseline generation completed successfully for all 164 problems, but the feedback generation and gating phase stalled without producing output files. The hypothesis remains untested. This report documents the implementation architecture and failure analysis to inform future work requiring infrastructure hardening for combined large model inference and dynamic code execution.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Large language models can explain code errors in detail, describing why an algorithm handles edge cases incorrectly or why a data structure choice leads to inefficiency. Yet these explanations are often incorrect. Analysis of Self-Refine by Madaan et al. (2023) indicates that 94% of its failures trace to bad feedback: 33% identify the wrong location and 61% propose incorrect fixes. Meanwhile, execution-based feedback from unit tests provides ground-truth correctness signals but offers no insight into why code fails—only that it fails.

This work frames the tension as one between **signal fidelity** and **semantic richness**. Execution feedback achieves high fidelity (tests are deterministic) but low richness (no explanation). AI feedback achieves high richness (detailed explanations) but low fidelity (frequent errors). These dimensions are orthogonal: high fidelity does not preclude high richness.

The proposed mechanism, **Execution-Verified AI Feedback (EVAF)**, uses execution to verify AI feedback rather than replace it. Given code that fails tests, an AI model generates a critique and proposed fix. The fix is executed against unit tests. If all tests pass, the critique is accepted as verified feedback; otherwise, it is rejected.

The **accept rate**—the fraction of AI suggestions surviving verification—determines EVAF’s viability. If accept rate is below 10%, EVAF degenerates to pure execution feedback because AI is nearly always wrong. If accept rate exceeds 90%, gating adds no value because AI is nearly always correct. The target range is 20-60%, indicating selective filtering.

This work implements EVAF using CodeT5-770M and CodeLlama-7b-Instruct on HumanEval. The contribution is architectural: a working implementation of the EVAF pipeline. However, the experimental evaluation did not complete due to infrastructure failure during execution. No metrics were computed, and the hypothesis remains untested.

2. Related Work

2.0.1. 2.1 EXECUTION-BASED FEEDBACK FOR CODE GENERATION

Execution feedback uses unit test outcomes as training signals. CodeRL (Le et al., 2022) applies actor-critic reinforcement learning for code, using test pass/fail as episode-level rewards. A critic network predicts functional correctness, enabling critical sampling for code regeneration.

RLTF (Liu et al., 2023) extends execution feedback with multi-granularity signals. Beyond binary pass/fail, RLTF extracts fine-grained error locations from unit test output, providing line-level localization. Online RL with real-time data generation is reported to yield strong results on APPS and MBPP benchmarks.

PPOCoder (Shojaee et al., 2023) combines PPO-based RL with non-differentiable execution feedback and structure alignment.

These methods share a characteristic: execution feedback provides no semantic guidance about why code fails.

2.0.2. 2.2 AI-GENERATED FEEDBACK FOR CODE

AI feedback uses LLM-generated critique for code improvement. Self-Refine (Madaan et al., 2023) demonstrates zero-shot iterative refinement: the same LLM generates code, critiques it, and refines based on its own feedback. No training is required—refinement occurs at inference time.

However, AI feedback has low fidelity. Madaan et al. analyze Self-Refine failures: 33% stem from identifying the wrong error location, and 61% from proposing incorrect fixes.

2.0.3. 2.3 GAP

No prior work uses execution as a verification mechanism for AI feedback during training. Execution and AI feedback have been developed as alternatives rather than combined.

3. Methodology

3.0.1. 3.1 FIDELITY AND RICHNESS FRAMEWORK

Feedback quality is decomposed into two dimensions:

Signal Fidelity: The probability that feedback is factually correct. Execution feedback achieves near-perfect fidelity (tests are deterministic). AI feedback has lower fidelity (approximately 39% correctness

based on Self-Refine analysis).

Semantic Richness: The amount of explanatory content about why code is wrong. Execution feedback has minimal richness (only pass/fail). AI feedback provides detailed explanations.

These dimensions are orthogonal. A method achieving high fidelity and high richness requires filtering rich signals through a fidelity check.

3.0.2. 3.2 EVAF MECHANISM

EVAF treats execution as a verification layer for AI feedback:

1. **AI Critique Generation:** Given problem description and failing code, the feedback model generates critique and proposed fix using a structured prompt.
2. **Code Extraction:** Fenced code blocks (“python ...”) are parsed from the AI response using regular expression matching.
3. **Execution Gating:** The extracted fix is executed against unit tests in a subprocess with 3.0 second timeout and network isolation.
4. **Accept/Reject Decision:** If all tests pass, the suggestion is accepted; otherwise, it is rejected with classified error type.

3.0.3. 3.3 IMPLEMENTATION ARCHITECTURE

Seven Python modules implement the pipeline:

- **config.py:** Configuration constants including model identifiers, generation parameters, and paths.
- **data.py:** HumanEval loading via HuggingFace datasets (`openai\_humaneval`).
- **model.py:** `BaselineModel` wrapping `CodeT5-770M` (`Salesforce/codet5-large`) and `FeedbackModel` wrapping `CodeLlama-7b-Instruct` (`codellama/CodeLlama-7b-Instruct-hf`, loaded in `float16`).
- **gating.py:** Code extraction using regex, error classification, and sandboxed subprocess test execution.
- **metrics.py:** Accept rate, coverage, and rejection breakdown computation.

- **visualize.py**: Figure generation (gate metrics bar chart, accept distribution histogram, rejection breakdown pie chart).
- **train.py**: Pipeline orchestration combining all modules.

3.0.4. 3.4 ACCEPT RATE AS VIABILITY INDICATOR

Accept Rate	Interpretation
<10%	EVAF degenerates to pure execution feedback; AI nearly always wrong
20-60%	Selective filtering; AI provides substantive verified feedback
>90%	Gating unnecessary; AI nearly always correct

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTION

Can EVAF produce filtered AI feedback with accept rate in the 20-60% range on HumanEval?

4.0.2. 4.2 DATASET

HumanEval: 164 Python programming problems with unit tests, loaded via HuggingFace datasets (`openai\_humaneval`, test split).

4.0.3. 4.3 MODELS

- **Baseline Generator**: CodeT5-770M (`Salesforce/codet5-large`), encoder-decoder architecture, 770M parameters.
- **Feedback Generator**: CodeLlama-7b-Instruct (`codellama/CodeLlama-7b-Instruct-hf`), decoder-only architecture, 7B parameters, loaded in float16 with `device_map="auto"`.

4.0.4. 4.4 GENERATION PARAMETERS

Parameter	Value
Baseline decoding	Greedy (<code>do_sample=False</code>)
Feedback temperature	0.2
Feedback max_tokens	512
Test timeout	3.0 seconds
Random seed	42

4.0.5. 4.5 EVALUATION PROTOCOL

1. Generate baseline code for all 164 HumanEval problems
2. Execute baseline code against tests; filter to failing problems
3. For each failing problem, run EVAF gating (generate feedback, extract code, execute tests)

4. Compute accept rate and secondary metrics
5. Evaluate gate condition

4.0.6. 4.6 SUCCESS CRITERIA

Outcome	Condition
PASS	Accept rate 20-60%
FAIL	Accept rate <10% or >90%
MARGINAL	Accept rate 10-20% or 60-90%

5. Results

5.0.1. 5.1 EXPERIMENT STATUS

Status: INCOMPLETE (infrastructure failure)

Stage	Status	Progress
Baseline generation	COMPLETE	164/164
Failure filtering	COMPLETE	164/164 failing
EVAF gating	INCOMPLETE	78/164 (48%) in one log; 87/164 (53%) in another
Metrics computation	NOT REACHED	—
Figure generation	NOT REACHED	—

5.0.2. 5.2 TIMELINE

- **Experiment start**: 2026-08-18T14:43:24Z
- **Baseline generation complete**: 3.5 minutes (164 problems at 1.2s/problem average)
- **EVAF gating progress**: 6.5 minutes for 78 problems (at 5s/problem average)
- **Termination**: Process stalled; no error message captured in logs

5.0.3. 5.3 PARTIAL OBSERVATIONS

From experiment logs:

- Baseline generation succeeded for all 164 problems
- All 164 baseline solutions failed at least one test (found 164/164 failing problems)
- EVAF gating processed 78-87 problems before termination
- No results.json, metrics.json, or figures were generated

5.0.4. 5.4 FAILURE ANALYSIS

No error message was captured. Hypothesized causes:

1. **GPU Memory Exhaustion (estimated likelihood: HIGH):** CodeLlama-7b-Instruct in float16 requires approximately 14GB GPU memory. Combined with CodeT5-770M and test execution overhead, memory pressure may have caused silent failure.

1. **Subprocess Timeout Escalation (estimated likelihood: MEDIUM):** Generated code containing infinite loops or long-running operations could cause subprocess timeouts to accumulate.

1. **System-Level Kill (estimated likelihood: MEDIUM):** External OOM killer or resource manager may have terminated the process.

5.0.5. 5.5 GATE VERDICT

Gate not evaluable: No accept rate metric available. The MUST_WORK gate for H-E1 cannot be assessed.

5.0.6. 5.6 ARTIFACTS

Artifact	Status
config.py	Complete
data.py	Complete
model.py	Complete
gating.py	Complete
metrics.py	Complete
visualize.py	Complete
train.py	Complete
results.json	Not generated
metrics.json	Not generated
Figures	Not generated

6. Discussion

6.0.1. 6.1 WHAT THE FAILURE DOES NOT INDICATE

The incomplete execution does not indicate:

- That EVAF does not work
- That accept rates are outside the viable range

- That the fidelity $\times$ richness framework is incorrect

The hypothesis remains untested. Infrastructure failure prevented measurement, but does not constitute theoretical refutation.

6.0.2. 6.2 WHAT THE FAILURE DOES INDICATE

Infrastructure requirements for EVAF pipelines:

1. Sufficient GPU memory for concurrent inference of 7B+ parameter models
2. Robust error handling and checkpointing for long-running experiments
3. Memory profiling and monitoring during execution
4. Resume capability after partial completion

Computational characteristics observed:

- Baseline generation: 1.2 seconds per problem on average
- EVAF gating: 5 seconds per problem on average (2-3x slower than baseline)
- Total estimated runtime for 164 problems: 15-20 minutes

6.0.3. 6.3 LIMITATIONS

1. **Single model pair:** Only CodeT5-770M + CodeLlama-7b-Instruct tested
2. **Single benchmark:** HumanEval only; no MBPP or APPS
3. **No training experiments:** Only accept rate measurement attempted
4. **No baseline comparison:** EVAF not compared to Exec-Fine or AI-Only conditions
5. **Incomplete execution:** No quantitative results available

6.0.4. 6.4 INFRASTRUCTURE RECOMMENDATIONS FOR FUTURE WORK

1. **Add checkpointing:** Save intermediate results after each problem to enable resume
2. **Profile GPU memory:** Monitor memory usage during CodeLlama inference

-
3. **Consider quantization:** 4-bit quantization could reduce memory requirements
 4. **Reduce batch processing:** Process problems sequentially with explicit memory cleanup
 5. **Add watchdog timers:** Detect and log hangs in subprocess execution

7. Conclusion

This work proposed Execution-Verified AI Feedback (EVAF), a mechanism combining execution verification with AI feedback for code generation. The theoretical contribution is the fidelity $\times$ richness framework: execution and AI feedback operate along orthogonal dimensions, and EVAF uses execution as a gating mechanism for AI feedback.

An implementation was developed comprising seven Python modules. The pipeline successfully loaded HumanEval (164 problems), CodeT5-770M, and CodeLlama-7b-Instruct. Baseline generation completed for all problems. However, the EVAF gating phase terminated at approximately 50% completion without producing output files.

What this work contributes:

- The fidelity $\times$ richness framework as a theoretical lens
- A complete EVAF implementation using standard tools
- Documentation of infrastructure requirements for future work

What this work does not contribute:

- Evidence that accept rate is in the viable range
- Evidence that verified feedback improves learning
- Comparison to baseline methods

The research question—can execution verify AI feedback at viable accept rates?—remains unanswered. Future work should address infrastructure stability before attempting experimental validation.

References

Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. Codet: Code generation with generated tests. In *International Conference on Learning Representations*, 2023.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Guo, S., Zhang, B., Liu, T., et al. Direct language model alignment from online ai feedback. *arXiv preprint arXiv:2402.04792*, 2024.

Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, pp. 21314–21328, 2022.

Liu, J., Zhu, Y., Xiao, K., Fu, Q., Han, X., Yang, W., and Ye, D. Rltf: Reinforcement learning from unit test feedback. *Transactions on Machine Learning Research*, 2023.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Shojaee, P., Jain, A., Tipirneni, S., and Reddy, C. K. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2301.13816*, 2023.

Wang, Y., Wang, W., Joty, S., and Hoi, S. C. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. pp. 8696–8708, 2021.

Zhou, C., Zhang, X., Song, D., et al. Refinecoder: Iterative improving via adaptive critique refinement. *arXiv preprint arXiv:2502.09183*, 2025.